$$G(z) = \frac{z^{-1}(9 + 6z^{-1} + 6z^{-2})}{(1 - 0.8z^{-1})(1 - 0.68z^{-1})(1 - 0.5z^{-1})} = \frac{z^{-1}(9 + 6z^{-1} + 6z^{-2})}{1 - 1.98z^{-1} + 1.284z^{-2} + 0.272z^{-3}}$$

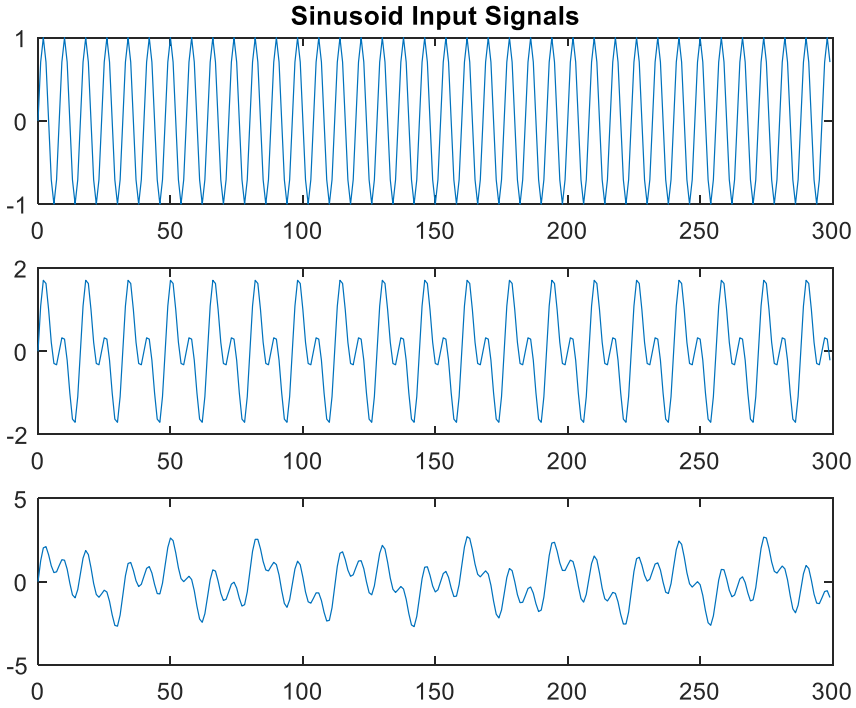
For this project, I used the same transfer function as in Project 1. This defined the plant I used for most of the calculations in this project.

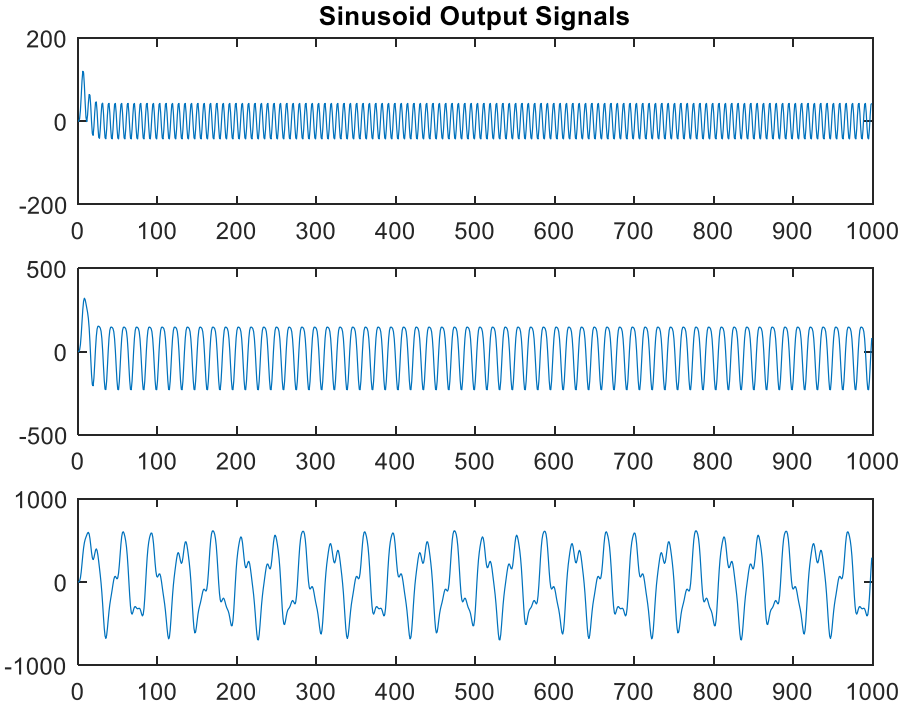
Question 1: Drive the Plant with Sinusoids

With my plant and Batch Least Squares estimation defined from project one, I was able to generate my sinusoidal inputs for the three sinusoids I used. After defining my sinusoids, I used the lsim command in MATLAB to drive my plant with sinusoid inputs and used my BatchLeastSquares function to estimate the parameters of the plant using the sinusoidal inputs and outputs over 1000 samples. The BatchLeastSquares function was slightly modified to also return the condition number of the data matrix $\Phi'\Phi$, but otherwise is the same as in project 1. The code for question 1 is shown below.

```
% 1. Drive plant witn one sinusoid
sin_input = sin(2*pi*(1/8)*t);
sin_output = lsim(sys, sin_input, t);
[BLS_sin, BLS_sin_cond] = BatchLeastSquares(sin_input, sin_output, t);

% 1. Drive plant with sun of two sinusoids
two_sin_input = sin(2*pi*(1/8)*t) + sin(2*pi*(1/16)*t);
two_sin_output = lsim(sys, two_sin_input, t);
[BLS_two_sin, BLS_two_sin_cond] = BatchLeastSquares(two_sin_input, two_sin_output, t);

% 1. Drive plant with sun of three sinusoids
three_sin_input = sin(2*pi*(1/8)*t) + sin(2*pi*(1/16)*t) + sin(2*pi*(1/38)*t);
three_sin_output = lsim(sys, three_sin_input, t);
[BLS_three_sin, BLS_three_sin_cond] = BatchLeastSquares(three_sin_input, three_sin_output, t);
```

The resulting input and output signals of these three simulations are shown below.





The above code produces parameter estimates for a single sinusoid, the sum of two sinusoids and the sum of three sinusoids as well as the condition numbers for those three estimates. The estimates and condition numbers are shown below. With a single sinusoid input on the left and the sum of three sinusoids on the right.

-1.9800	-1.9800	-1.9800
1.2840	1.2840	1.2840
-0.2720	-0.2720	-0.2720
9.0000	9.0000	9.0000
6.0000	6.0000	6.0000
6.0000	6.0000	6.0000
4.3855e-10	-3.3710e-09	-8.0542e-09
9.4895e+07	5.9975e+08	2.4339e+09

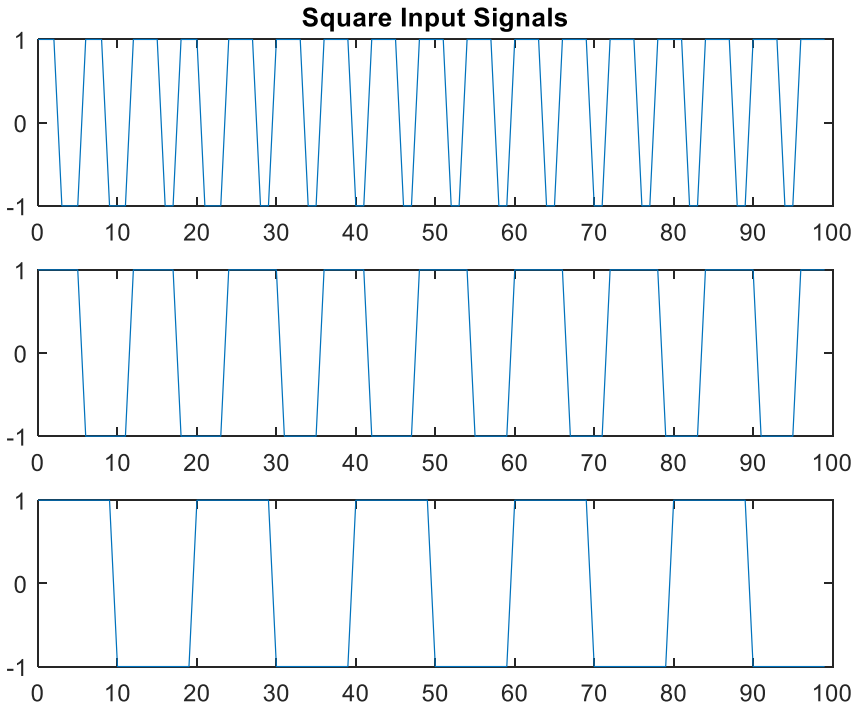
For each of these estimates, the batch least squares estimation comes out to be very close to the expected parameter estimates. The condition numbers increase significantly as the number of sinusoids in the input also increases. I believe the estimates are very accurate due to the large number of samples used, which leads to a more accurate estimation. Additionally, I believe the increasing condition numbers to be caused by the larger periods in the resultant sinusoidal inputs.

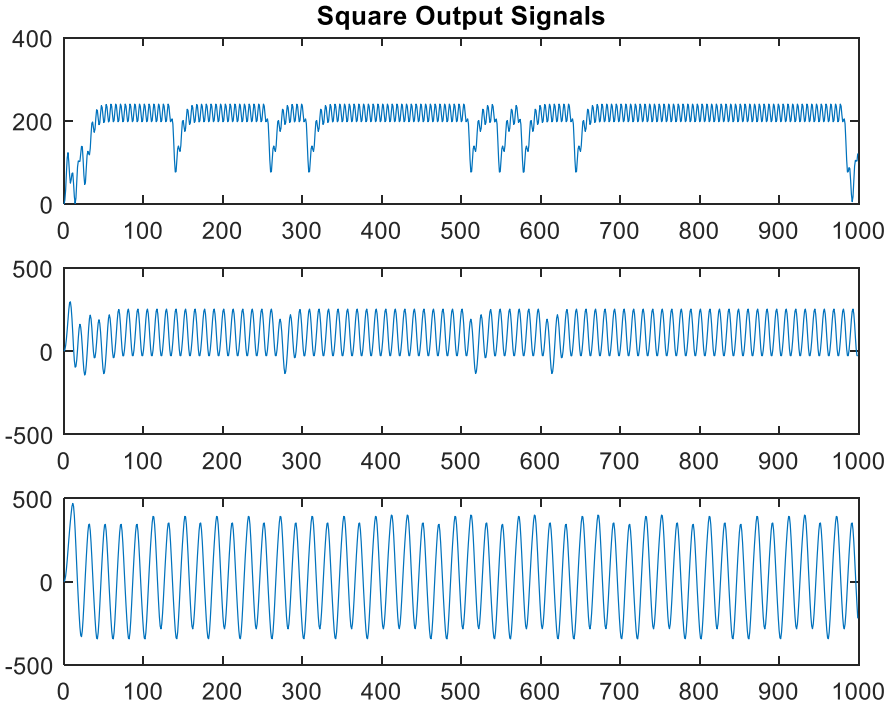
Question 2: Drive plant with Symmetrical Square-Waves

For this question, I used a similar method to build my square wave inputs as was used in question 1. However, instead of using sin() to generate the inputs, I used square() to generate the inputs. With inputs generated, I then simulated the system with each input and used the batch least squares estimation function to estimate the parameters and find the condition number. The code to do this is shown below.

```
% 2. Square wave with a period of 6
period = 6;
sq_input = square(2 * pi * (1 / period) * t);
sq_output = lsim(sys, sq_input, t);
[BLS_sq, BLS_sq_cond] = BatchLeastSquares(sq_input, sq_output, t);

% 2. Square wave with a period of 12
period = 12;
sq_input_12 = square(2 * pi * (1 / period) * t);
sq_output_12 = lsim(sys, sq_input_12, t);
[BLS_sq_12, BLS_sq_cond_12] = BatchLeastSquares(sq_input_12, sq_output_12, t);

% 2. Square wave with a period of 20
period = 20;
sq_input_20 = square(2 * pi * (1 / period) * t);
sq_output_20 = lsim(sys, sq_input_20, t);
[BLS_sq_20, BLS_sq_cond_20] = BatchLeastSquares(sq_input_20, sq_output_20, t);
```

This generates the inputs, outputs, and estimations for the three cases laid out in question two. The input and output signals generated by this code is graphed below.





With these input and output signals generated, I was able to pass these signals into my BatchLeastSquares function. These are the resulting estimations and condition numbers, with the square wave with a period of 6 on the left and the square wave with the period of 20 on the right.

-1.9800	-1.9800	-1.9800
1.2840	1.2840	1.2840
-0.2720	-0.2720	-0.2720
9.0000	9.0000	9.0000
6.0000	6.0000	6.0000
6.0000	6.0000	6.0000
-2.3142e-09	-2.4968e-10	-4.5339e-10
4.3948e+07	1.0806e+07	2.6174e+07

These three estimations come out to be very accurate to actual parameters, which I believe is also due to thousand samples used which gives enough excitation of the system to estimate the parameters. The condition number for all three is in the range of 1e7, which is similar to the condition number of the single sinusoid from question one but still acceptable by the requrements laid out in the assignment.

Question 3: Sliding Window with Step Input

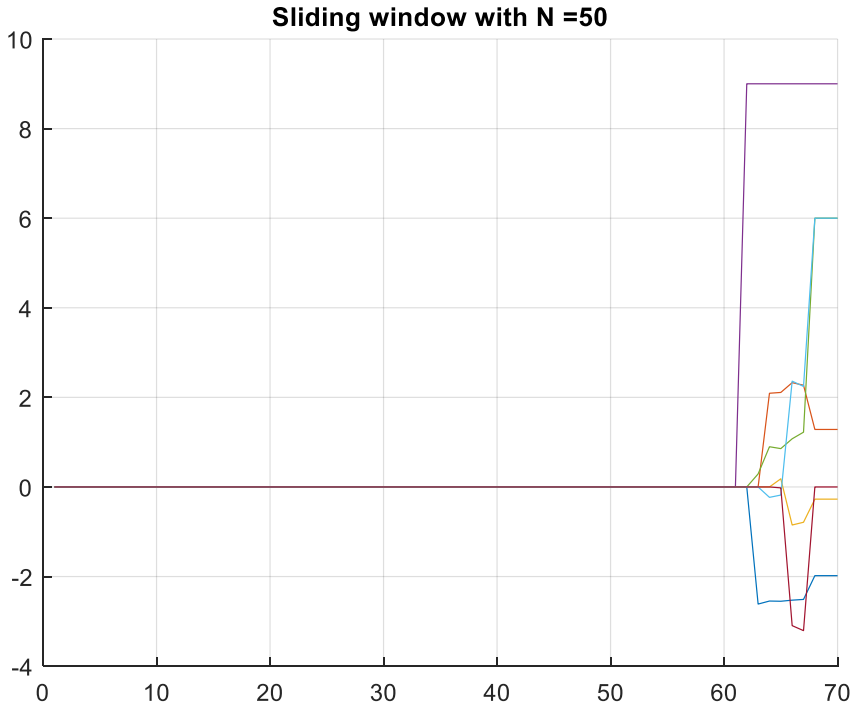
For this question, I had to generate a step input that occurs at the 60th sample, which I did manually by concatening a matrix of 60 zeros and 10 ones. This input was then used to simulate the system and this input and output was passed into the sliding window algorithm with a window size of 50. The sliding window algorithm is identical to project one, with the only changes being returning the condition number and using pinv() to generate the inverse matrix instead of using inv(). This change was implemented to handle errrors where matrices weren't invertible and produced NaN instead of estimations of the parameters. The code for this is shown below.

```
% 3. Sliding window with step input
t_win = 1:1:70;
step_input = horzcat(zeros(1,60), ones(1,10));
step_output = lsim(sys, step_input, t_win);
[SW_output, SW_cond_num] = SlidingWindow(step_input, step_output, 50, t_win, 4);
```

This generates the ending estimate (left) and condition number (right)shown below.

-1.9800	
1.2840	
-0.2720	
9.0000	
6.0000	
6.0000	
-7.5453e-10	1.5312e+08

The graph of the parameter estimations over the 70 samples is shown below.



As the estimation graph shows, all estimations are at zero as the input remains at 0. Due to there being no excitation of the system during the first 60 inputs, there is no output that can give a good estimation. Once the input begins to excite the system, the sliding window estimation is able to estimate the parameters quickly and accurately. The condition number of the data matrix is within the acceptable range.

Question 4: Overparameterize the System

For the fourth question, I drove the system with random input and overparameterized the system. For this, I created a second batch least squares function, with the ability to take in a changing number of numerator and denominator parameters. This function is shown below.

```
function [ theta, cond_num ] = BatchLeastSquaresV2 ( input, output, np, dp, time)
% 3 Batch LS program to identify the parameters
numSamples = length(time);

% Create the phi matrix based
for i=np:numSamples
    for j=1:dp
        if(i-j <=0)
            phi(i,j)=0;
        else
            phi(i,j)=-output(i-j);
        end
    end
    for j=0:np
        if(i-j-1 <=0)
            phi(i,j+dp+1)=0;
        else
            phi(i,j+dp+1)=input(i-j-1);
        end
    end
end

% Calculate Batch Parameters
theta =inv(phi' * phi)*phi'*output;
cond_num = cond(phi' * phi);
```

With this new function defined and taking in the number of parameters in the numerator and denominator, I built a random input, simulated the system for a random output and then estimated the parameters of this system, but overparameterized in the ways described in the assignment. The code for this is shown below.

```
% Random input and subsequent output for reference
rand_input = rand(n,1) - 0.5;
rand_output = lsim(sys, rand_input, t);
```

```
% 4. Operparameterize numerator
[BLS_OP_num, BLS_OP_num_cond] = BatchLeastSquaresV2(rand_input, rand_output,4,3, t);

% 4. Operparameterize denomenator
[BLS_OP_den, BLS_OP_den_cond] = BatchLeastSquaresV2(rand_input, rand_output,3,4, t);

% 4. Operparameterize both
[BLS_OP_both, BLS_OP_both_cond] = BatchLeastSquaresV2(rand_input, rand_output,4,4, t);
```

For each of these overparameterized estimates, the following parameter estimates, and condition numbers are generated. The estimates shown are for overparameterization in the numerator, in the denominator and in both respectively.

		-0.9548
-1.9800	-1.0172	-0.4317
1.2840	-0.6789	0.9543
-0.2720	1.2964	-0.2698
9.0000	-0.2682	9.0000
6.0000	9.0000	15.0820
6.0000	15.6117	11.8402
5.0261e-10	12.4848	5.4507
1.6097e-10	6.4473	-3.6527e-12
2.7700e+07	2.9869e+17	5.5004e+17

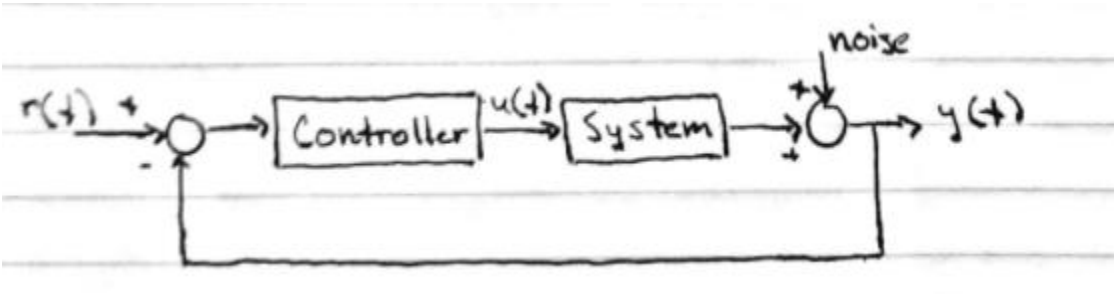
My observations for each one of these estimates is below.

- Overparameterize the numerator: When the numerator is overparameterized, the estimate remains mostly the same. The estimation grains one additional zero in the numerator's estimation but the remainder of the estimations remain the same. The condition number of the data matrix is within acceptable ranges.
- Overparameterize the denominator: When the denominator is overparameterized, almost every estimate is rendered incorrect. The denominator estimations are entirely incorrect, but the numerator estimate does have one correct value. In this instance, we experience pole-zero cancellation and the system remain unidentified. The condition matrix is significantly out of acceptable ranges.
- Overparameterize both: When both numerator and denominator are overparameterized, the estimate is again almost entirely incorrect. The numerator does get some estimated values close to target values, but the estimate as a whole is incorrect. The condition matrix is significantly out of acceptable ranges.

Overall, the estimations are rendered greatly inaccurate anytime that the denominator is overparameterized due to pole-zero cancellation. The case of overparameterized numerator remains accurate, but in all of these cases, the condition number is representative of whether or not the estimate can be believed to be reasonably accurate.

Question 5: Feedback System

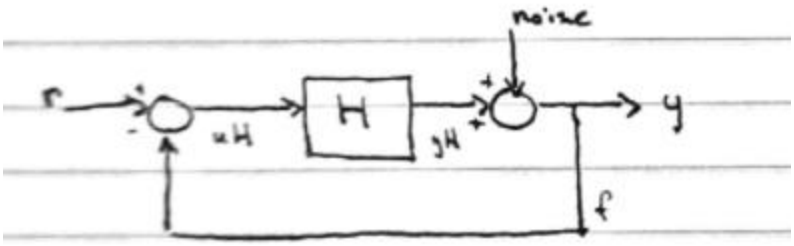
For this question, I was given a feedback system that I had to simulate. The feedback system is drawn below.



To build and simulate this system, I built a function that takes in the controller, input and noise and will link these signals and transfer functions as shown above. The code for this function is shown below.

```
function [theta, cond_num] = ClosedLoopSystem(Gc, Gp, input, noise, t, graphNum)

% Define some variables for future use
H = Gc * Gp;
r = input;

% Construct my feedback system per the project instructions
Sum2 = sumblk('f = yH + n');
Sum1 = sumblk('uH = r - f');
H.u = 'uH';  H.y = 'yH';
T=connect(H,Sum1, Sum2, {'r','n'} , 'f');

% Simulate system for all inputs
y = lsim(T, [r,noise]);

% Re-simulate inputs with controller system to calculate u for the system
for i=1:length(y)
    d(i) = r(i) - y(i);
end
u = lsim(Gc,d);

% Estimate plant parameters using Batch Least Squares method
[theta, cond_num] = BatchLeastSquares(u,y,t);
```

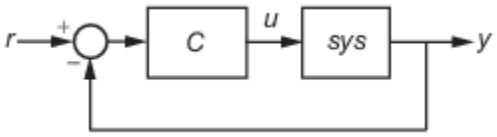
To link these systems together, I took advantage of the connect function within MATLAB as well as the sumblk function to create the nodes and plants within this feedback loop. For simplicity, I merged the controller and system into one transfer function named H and then defined my summation nodes and connected the summation nodes, plant, noise and input together. The simplified system is drawn below with labels so that the structure used can be better displayed.



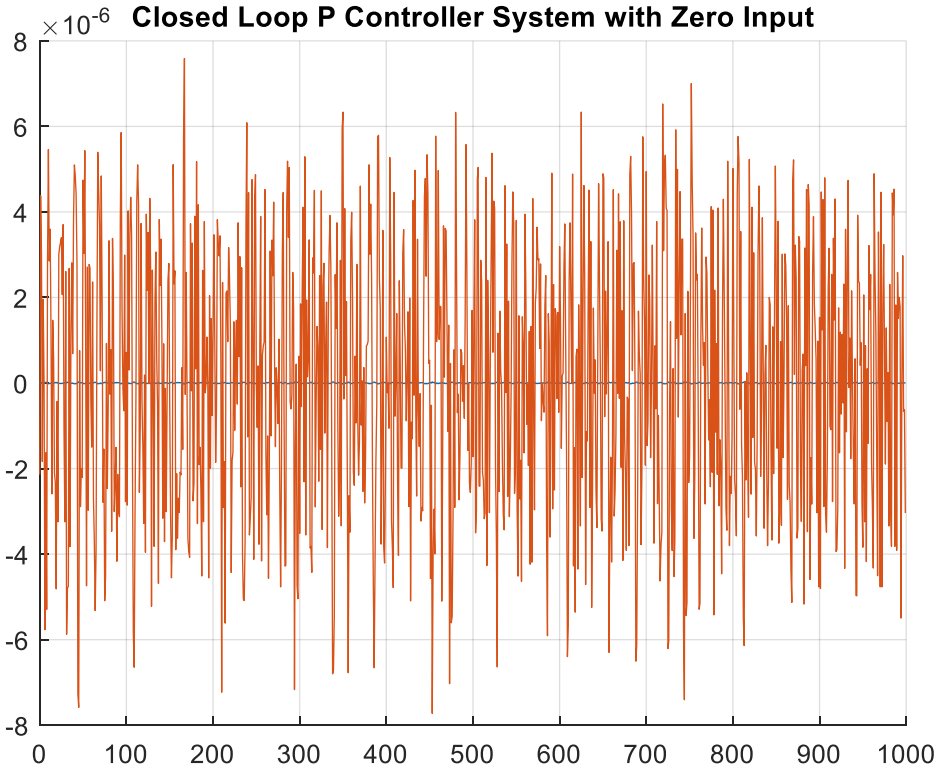
Once the system was built to the specs outlined, it was saved as a two input one output transfer function 'T'. This simulated was then simulated using the lsim command, taking both r(t) and the random noise as inputs. Once the overall system was simulated, I was able to simulate the controller with r(t) – y(t) as the input in an open loop configuration, allowing me to measure u(t) for the run of the simulation. With the system simulated and y(t) & u(t) measured, I estimated the parameters of my plant using the batch least squares method. This was done for all four cases outlined in the assignment, using zero input or random input with either a P or PI controller. The code for running this function is shown below.

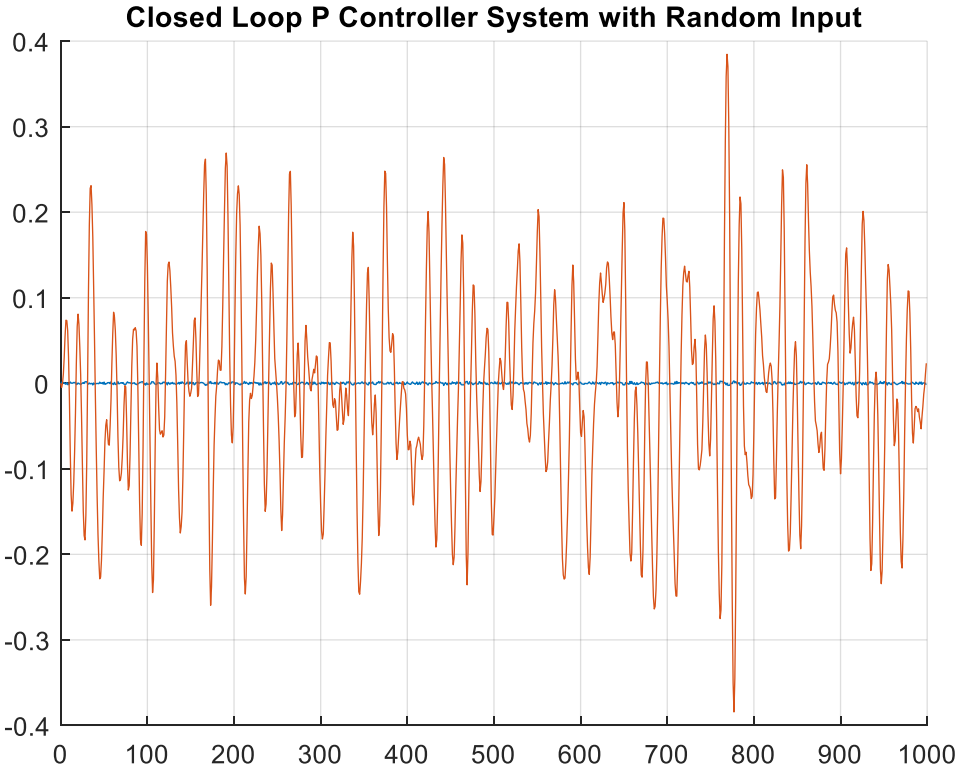
```
% 5. Simulate the system with feedback
Gp = sys;
zero_input = zeros(1,n)';

% P controller with zero input
Gc = pidtune(sys, 'P');
[feedback_theta_p1, feedback_cond_num_p1] = ClosedLoopSystem(Gc, Gp, zero_input, noise, t, 6);

% P controller with random input
[feedback_theta_p2, feedback_cond_num_p2] = ClosedLoopSystem(Gc, Gp, rand_input, noise, t, 7);

% PI controller with zero input
Gc = pidtune(sys, 'PI');
[feedback_theta_pi1, feedback_cond_num_pi1] = ClosedLoopSystem(Gc, Gp, zero_input, noise, t, 8);

% PI controller with random input
[feedback_theta_pi2, feedback_cond_num_pi2] = ClosedLoopSystem(Gc, Gp, rand_input, noise, t, 9);
```

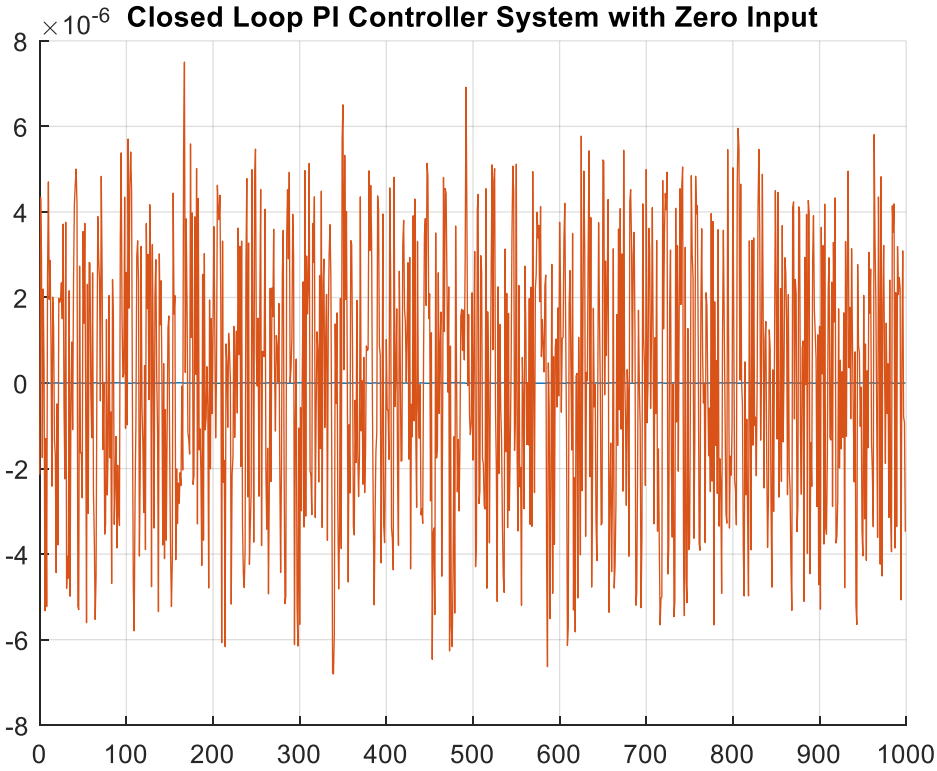
To get an effective P/PI controller, I leveraged the pidtune() command within MATLAB. This command, given a system and a controller type will build a tuned controller for the system given a very simple closed loop system. This closed loop system that pidtune() uses is shown below.

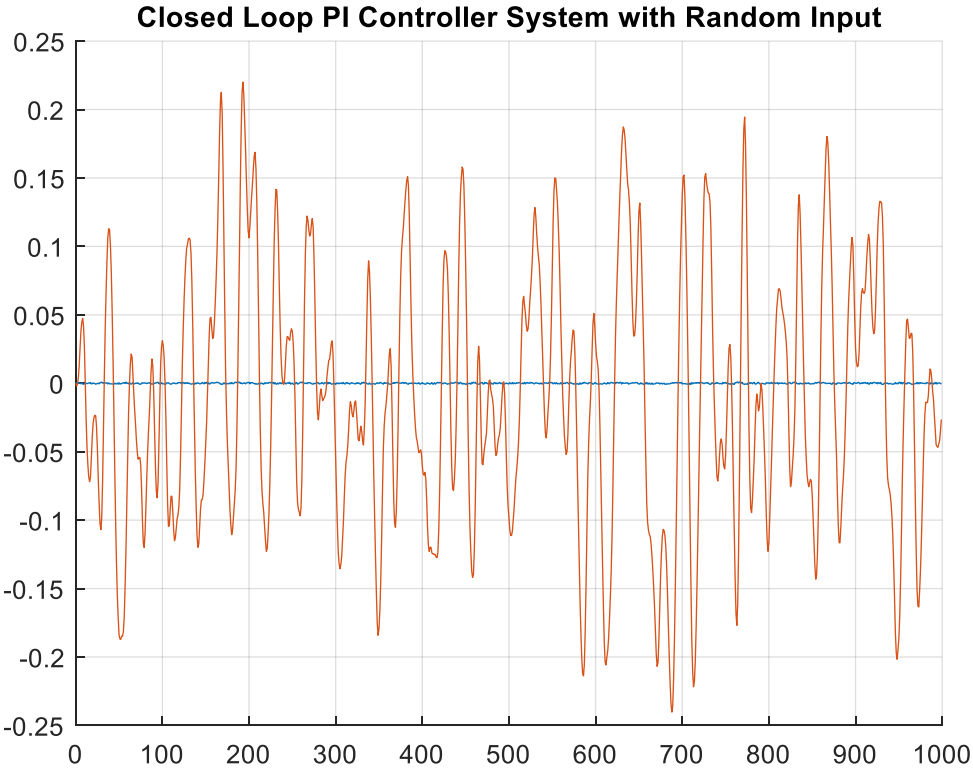


This allowed me to create and tune a controller for my system, and then run the systems as shown above. The results for these simulations are shown below. The orange line in the system is the u value, and the blue line is the y value


Closed Loop P Controller System with Zero Input
×10^-6
8
6
4
2
0
-2
-4
-6
-8
0
100
200
300
400
500
600
700
800
900
1000


Closed Loop P Controller System with Random Input
0.4
0.3
0.2
0.1
0
-0.1
-0.2
-0.3
-0.4
0
100
200
300
400
500
600
700
800
900
1000


Closed Loop PI Controller System with Zero Input
×10^-6
8
6
4
2
0
-2
-4
-6
-8
0
100
200
300
400
500
600
700
800
900
1000


Closed Loop PI Controller System with Random Input
0.25
0.2
0.15
0.1
0.05
0
-0.05
-0.1
-0.15
-0.2
-0.25
0
100
200
300
400
500
600
700
800
900
1000

In each of these simulations, the controller aims to keep the system output at a zero mean which is by design of the pidtune() function. It is possible to use the pidtune() function to aim for another value to tune the system to, but I chose to leave it to the default value. In the cases with zero input, the system is driven exclusively by the noise of the system, which is what causes such a 'noisy' u value. With the random input, the controller is able to seemingly smooth the input, which gives me the ability to properly estimate the parameters. The final estimations and condition numbers of the four cases are shown below.

P Controller (zero, random):

-0.1209	-1.9801
-0.0238	1.2842
0.1127	-0.2721
-4.1889e-04	9.0002
-8.2436e-05	5.9983
3.9041e-04	5.9992
35.7696	-0.0019
1.1364e+21	1.5912e+06

PI Controller (zero, random):

-0.3507	-1.9803
-0.3300	1.2845
-0.2850	-0.2722
212.6000	9.0005
212.6001	5.9960
212.6002	5.9985
-31.4930	-0.0044
2.7460e+21	5.8014e+06

For both of these controller types, I found that the parameters were identifiable with the random input but not identifiable with the zero input. In the case of the zero input, there was not enough excitation in the system to identify the parameters correctly and this is reflected in both the estimation being very incorrect and the condition number of the data matrix being out of acceptable ranges. For the random input, the parameters were identifiable as there was sufficient excitation of the system to allow for parameters to be identified. However, I did notice that the system was quite susceptible to noise interfering with the estimation, much more than in a non-close loop system. I believe this to be the effect of the controller, as it does attempt to keep the output values in a small range, which causes noise in the feedback to change the inputs to the controller and plant which then affects the parameter estimation.

Overall, this project has shown me a lot regarding sufficient excitation of systems, the effects of overparameterization, and how a condition matrix can be used to check if the estimation can be believed to be correct.